

Tanque de Combustível

Objetivo: Monitorar o combustível do motor e detectar se há vazamentos perigosos no casco do tanque. É usado tanto pelo Motor de Propulsão (para acelerar) quanto pelo Canhão Laser (para carregar a arma).

TanqueCombustivel
- tipoCombustivel: String - volumeAtual: double - vazamentoDetectado: boolean + CONSUMO_PADRAO: double = 15.0 <static>
+ TanqueCombustivel(tipo: String) + getTipoCombustivel(): String + getVolumeAtual(): double + setVolumeAtual(volume: double): void + isVazamentoDetectado(): boolean + setVazamentoDetectado(status: boolean): void + toString(): String + equals(obj: TanqueCombustivel): boolean + injetarCombustivel(qtd: double): void + consumir(qtd: double): void + consumir(): void « sobrecarga » + verificarVazamento(): boolean

Regras:

1. Construtor: recebe `tipoCombustivel`. `volumeAtual` inicia em `1000.0`, `vazamentoDetectado` em `false`.
2. `toString`: informa tipo, volume e se há vazamento.
3. `equals`: assinatura `equals(TanqueCombustivel obj)`. Compara se o atributo `tipoCombustivel` é igual ao tipo de combustível do parâmetro.
4. `injetarCombustivel(double qtd)`: adiciona volume ao `volumeAtual`. Porém, se `vazamentoDetectado` for `true`, apenas 50% do valor injetado entra no tanque (o resto se perde no espaço).
5. `consumir(double qtd)`: se tentar consumir mais do que `volumeAtual`, o volume zera e `vazamentoDetectado` vira `true` (forçaram a bomba). Se for normal, apenas subtrai.
6. `consumir()` — **SOBRECARGA**: consome a quantidade padrão definida pela constante `CONSUMO_PADRAO` (15.0). Não deve duplicar a lógica de vazamento: internamente apenas delega para `consumir(CONSUMO_PADRAO)`. A constante é **pública** (visibilidade +) porque o `MotorPropulsao` consulta `TanqueCombustivel.CONSUMO_PADRAO` para saber, antes de acelerar, se há combustível suficiente para o consumo padrão.
7. `verificarVazamento()`: apenas retorna o status atual do vazamento.

Atenção: `consumir(double)` e `consumir()` são uma SOBRECARGA (overload) — mesmo nome, parâmetros diferentes, dentro da própria classe.